accenture Code.Hub

PROJECT MULTISALES:

End-to-End Data Engineering Pipeline

Team 3:

Bougiouri Maria
Karatzas Christos
Papageorgiou Dimitris
Loukas Anastasis

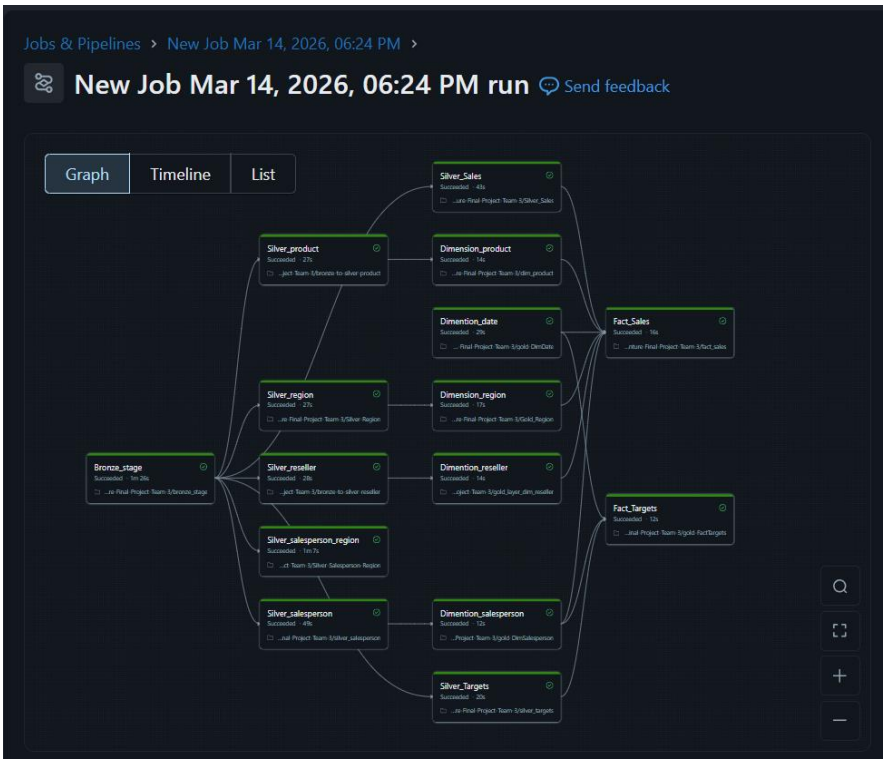
INTRODUCTION & TEAM COLLABORATION

For this project, our priority was to ensure everyone gained hands-on experience across the entire pipeline. Rather than splitting the work by role (e.g., one person doing only data ingestion and another only BI), everyone contributed to the Bronze, Silver, and Gold layers. We relied heavily on Discord for our daily syncs, pair programming, and debugging. To manage our codebase, we linked a shared GitHub repository directly to our Databricks workspace.

Github: https://github.com/Anastasis-Loukas/Accenture-Final-Project-Team-3

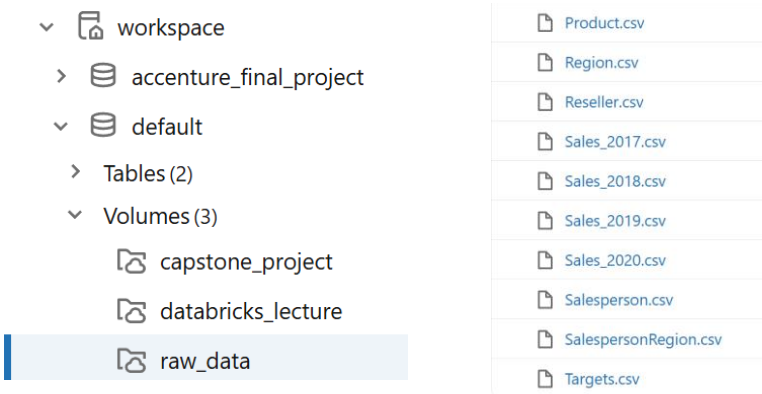
1. Evidence of Pipeline Execution and Outputs

This section presents the successful execution of the ETL Pipeline within Databricks, following the Medallion Architecture principles.



2. Bronze Layer (Raw Data Ingestion)

In the Bronze Layer, we ingested the initial raw data (CSVs 2017-2020, Products, Resellers, etc.) and stored them in their native state as Delta Tables. This ensures that we maintain an accurate, immutable history of the source files.



3. Silver Layer (Data Cleansing & Transformation)

In the Silver Layer, we applied our Data Quality rules. We removed duplicates, corrected data types (type casting), and handled missing (null) values to prepare the data for modeling.

```
4 days ago (<1s)

# Silver Layer Transformations
df_salesperson_silver = (df_salesperson_raw
    # Basic standardization of text fields: Trim whitespaces
    .withColumn("Salesperson", F.trim(F.col("Salesperson")))
    .withColumn("Title", F.trim(F.col("Title")))
    .withColumn("UPN", F.lower(F.trim(F.col("UPN"))))
    .dropDuplicates()
    .drop("_rescued_data")
)
```

4. Gold Layer (Business-Ready Model)

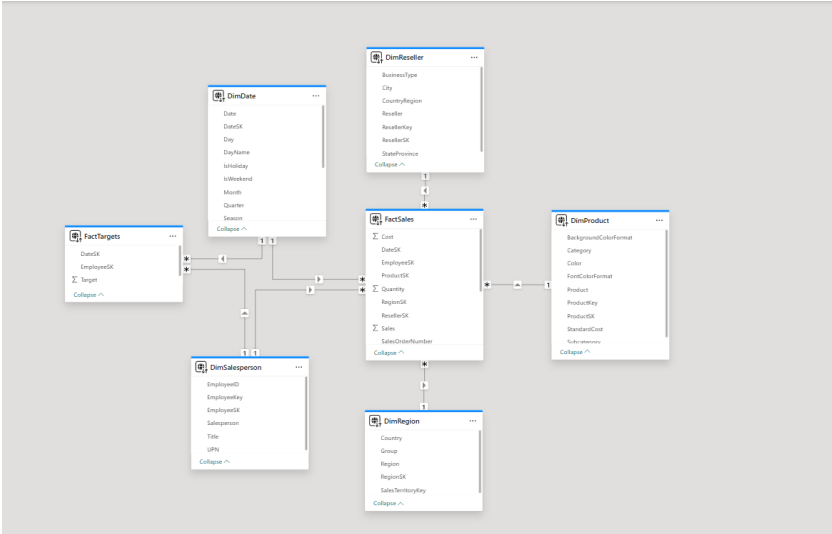
The Gold Layer contains our final Galaxy Schema (Fact and Dimension tables), highly optimized and ready for consumption by Power BI.

```
2 days ago (1s)                                                    2

#Join the tables to create the fact table
#We use left join to keep all targets
fact_targets = targets.join(dim_date,targets.TargetDate==dim_date.Date,"left")\
    .join(dim_salesperson,targets.EmployeeID==dim_salesperson.EmployeeID,"left")

#Select the columns we want to include in the fact_targets table
fact_targets = fact_targets.select("EmployeeSK","Target","DateSK")
display(fact_targets)
> See performance (1)
> fact_targets: pyspark.sql.connect.dataframe.DataFrame = [EmployeeSK: integer, Target: double ... 1 more field]
```

gold_layer
- dim_date
- dim_product
- dim_region
- dim_reseller
- dim_salesperson
- fact_sales
- fact_targets



5. Tables and Diagrams of Implementation

This section documents the specific data quality issues identified and the structure of the final dimensional model.

Data Quality Issues & Handling in Silver Layer

Dataset (Source)	Data Quality Issue Identified	Handling Decision in Silver Layer
Sales (2017-2020)	Records with Null `CustomerID` or `ProductID`.	Dropped rows, as sales without a valid product or customer are considered invalid.
Sales (2017-2020)	Presence of duplicate rows.	Applied the `.dropDuplicates()` method based on unique keys (e.g., `TransactionID` / `OrderDate`).
Products	Incorrect data types (e.g., `Price` read as String instead of Double).	Applied type casting using `.withColumn("Price", col("Price").cast(DoubleType()))`.

Description of the Gold Model (Tables and Grain)

The final architecture of the Gold Layer was designed based on Dimensional Modeling principles (Galaxy Schema), optimized for analytical query performance in Business Intelligence tools like Power BI.

Table Name	Type	Description & Grain
Fact_Sales	Fact Table	The central fact table for sales events. **Grain:** One record per individual sale (per day, per product, per salesperson). Contains metrics such as `SalesAmount` and `Quantity`.
Dim_Product	Dimension Table	**Grain:** One record per unique Product. Contains descriptive attributes like Category, Color, and Standard Cost.
Dim_Salesperson	Dimension Table	**Grain:** One record per unique Salesperson.
Dim_Region	Dimension Table	**Grain:** One record per Geographical Region (Country, State, Group).
Dim_Date	Dimension Table	**Grain:** One record per Date. Enables time-series analysis (Year, Quarter, Month).
Fact_Targets	Fact Table	**Grain:** One record per Salesperson per Month/Year. Contains historical sales targets.